JAVASCRIPT'IN KALBI

Event Loop & Asenkron Mimari

Tek iş parçacıklı bir dilin yüksek performanslı asenkron dünyasını keşfedin.

HAZIRLAYAN VE SUNAN
Ayşe Nurcihan Akbaş

PROJE TAKIMI
ALTAY

SUNUM AKIŞI

Bugün Neler Öğreneceğiz?

01

JS Engine & Single Thread

Yorumlanan yapı ve tek kanal mekanizması.

02

Senkron vs Asenkron

Sıralı adımlar ve arka plan operasyonları.

03

Call Stack (Yığın)

LIFO prensibiyle anlık yürütme mantığı.

04

Web APIs & Arka Plan

Tarayıcı yetenekleri ve zamanlayıcılar.

05

Task vs Microtask

Kuyruklar arası VIP öncelik sıralaması.

06

Event Loop & Örnekler

Konsol çıktıları ve pratik senaryolar.

</> TEMEL KAVRAMLAR

JavaScript Nasıl Bir Dildir?

Yorumlanan & Dinamik

Derlenmeye ihtiyaç duymadan tarayıcı üzerinde anlık olarak çalıştırılır.

Tek İş Parçacıklı

Aynı anda yalnızca tek bir işlem gerçekleştirir. Sırayla ilerler.

Davranış Katmanı

HTML ve CSS üzerine kullanıcının etkileşime geçtiği dinamik mantığı kurar.

⚠ MİMARİ ZORLUK

Single Thread Olmak Ne Anlama Gelir?



Kilitlenme Tehlikesi (Blocking)

Uzun süren veritabanı veya ağ sorgularında tüm sayfa donabilir, kullanıcı hiçbir şeye tıklayamaz.



Çözüm: Asenkron Mimari

Uzun işleri arka plana devredip ana kanalı açık tutmak. İşte Event Loop tam olarak bunu sağlar!

MANTIK KARŞILAŞTIRMASI

Senkron vs Asenkron Çalışma

SENKRON

Sıralı Adımlar (Bloklayıcı)

Her komut bir öncekinin bitmesini kesinlikle bekler.

1. Sipariş al
2. Yemeği pişir (Bekle...)
3. Masaya servis et
4. Sonraki müşteri

ASENKRON

Eşzamanlı / Arka Plan

Sipariş verilir, pişerken diğer müşterilerle ilgilenilir.

1. Siparişi mutfağa ilet
2. Sonraki müşteriyi al
3. Yemek pişince çağır
4. Servis yap

MİMARİ GENEL BAKIŞ

Event Loop Çalışma Alanı

1

Call Stack

Kodun fiilen çalıştırıldığı yığın. LIFO (Son giren ilk çıkar) mantığıyla çalışır.

2

Web APIs

Tarayıcının sunduğu arka plan bileşenleri (setTimeout, fetch, DOM events).

3

Event Loop

Stack boşaldığında kuyruktaki bekleyen işleri Stack'e taşıyan döngü mekanizması.

4

Task & Microtask Queues

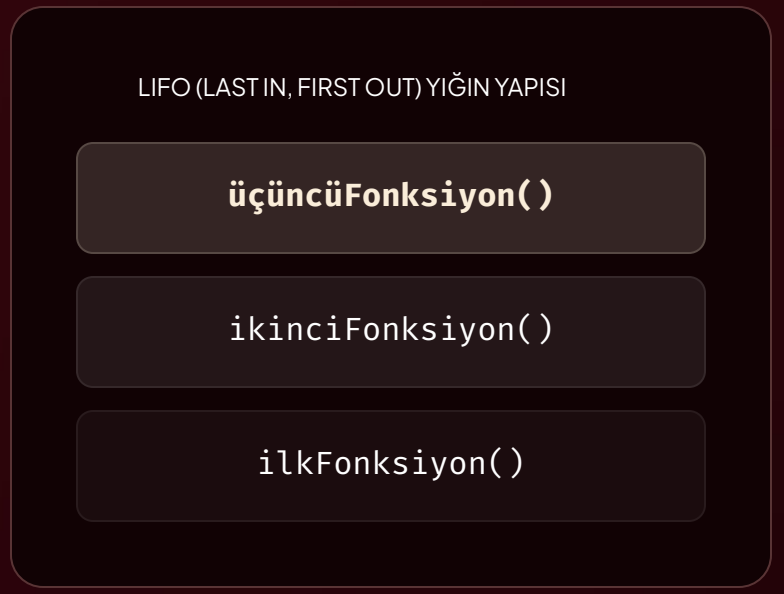
Süresi dolan asenkron görevlerin (Callback) çalışmak için sıraya girdiği alan.

DERİNLEMESİNE BİLEŞENLER

Call Stack (Çağrı Yığını)

JS engine'in "Şu an ne yapıyorum?" sorusuna yanıt verdiği yerdir.

- Fonksiyon çağrıldığında yığına eklenir (Push).
- Fonksiyon tamamlandığında yığından çıkarılır (Pop).
- Tek iş parçacıklı olduğu için sadece en üstteki işi yürütür.



TARAYICI GÜCÜ

Web APIs: Arka Plan Kahramanları

JavaScript Engine tek başınadır ancak Tarayıcı (Browser) ona geniş bir yardımcı kütüphane sunar.

Zamanlayıcılar

setTimeout, setInterval gibi fonksiyonların arka planda sayımını gerçekleştirir.

Ağ İstekleri

fetch() ve XMLHttpRequest ile sunucu haberleşmelerini yönetir.

DOM Olayları

onClick, onLoad gibi kullanıcı etkileşim dinleyicilerini arkada tutar.

ÖNCELİKLİ HAT VE KUYRUKLAR

Task Queue vs Microtask Queue

Task Queue (Callback Queue)

NORMAL HAT

Web API tarafında süresi dolan callback'ler sıraya girer. FIFO kuralı geçerlidir.

- Click Callback
- Timeout Callback

Microtask Queue

VIP GEÇİŞ

Promise, async/await ve queueMicrotask içerikleri buraya düşer.

Altın Kural: Event Loop, normal Task'e bakmadan önce HER ZAMAN Microtask'leri bitirir!

GÖRSEL ÖZET

Event Loop Çalışma Döngüsü

ADIM 1

Call Stack Çalışır

Senkron kodlar anında yürütülür, yığına sırasıyla girip çıkar.

ADIM 2

Microtask Önceliği

Stack boşalınca ilk olarak Microtask Queue kontrol edilir ve tüketilir.

ADIM 3

Task Queue Sırası

Microtask'ler bittikten sonra Task Queue içerisindeki zamanlayıcılar işlenir.

KOD ÜZERİNDE GÖRELİM

Konsol Çıktısı Sıralaması Nasıl Olur?

```
// Kod Analizi
console.log('1: Başlangıç');

setTimeout(() => {
   console.log('2: Timeout');
 }, 0);

Promise.resolve().then(() => {
   console.log('3: Promise');
 });

console.log('4: Bitiş');
```

ÇIKTI SIRALAMASI:

1. **1: Başlangıç** (Senkron)
2. **4: Bitiş** (Senkron)
3. **3: Promise** (Microtask VIP)
4. **2: Timeout** (Task Queue)

MODERN YAZIM

Async / Await ile Temiz Kod

Promise yapısının sunduğu asenkron gücü, sanki senkron kod yazıyormuş gibi okumamızı sağlar.

async Anahtar Kelimesi

Fonksiyonun her zaman otomatik olarak bir Promise döndürmesini sağlar.

await Anahtar Kelimesi

Promise çözülene kadar arka planda bekler, ancak ana thread'i asla tıkamaz!

DİKKAT EDİLMESİ GEREKENLER

Sık Yapılan Hatalar

Ağır Döngüler

Ağır for döngüleri Call Stack'i kilitler ve Event Loop'un çalışmasını durdurur.

Uncaught Promise

Asenkron hataları yakalamak için try...catch veya .catch() unutulmamalıdır.

Gereksiz Await

Bağımsız işleri ardışık bekletmek yerine Promise.all ile paralel çalıştırmak gerekir.

🏁 ÖZET

Neler Öğrendik?

- JS Single-Threaded bir dildir ancak Event Loop ile yüksek performanslı asenkron çalışır.
- Call Stack tamamen boşalmadan hiçbir asenkron görev çalıştırılmaz.
- Microtask Queue (Promises), normal Task Queue'ya kıyasla her zaman VIP önceliğe sahiptir.

TEŞEKKÜRLER

Dinlediğiniz İçin Teşekkürler!

Sorularınız varsa cevaplamaktan memnuniyet duyarım.

Sunan: Ayşe Nurcihan Akbaş | **Takım:** Altay